Task1

1.

```
package Task6;

import  java.io.File;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
public class Main {
    public static void main(String[] args) throws IOException {
        File file1 = new File("doro.jpg");
        File file2 = new File("doro_copy.jpg");
        FileInputStream fis1 = new FileInputStream(file1);
        FileOutputStream fis2 = new FileOutputStream(file2);
        byte[] b1 = new byte[1024];
        int len;
        while((len = fis1.read(b1)) != -1) {
            fis2.write(b1, 0, len);
        }
        fis2.close();
        fis1.close();

    }
}
```

Task2

```java
package Not;

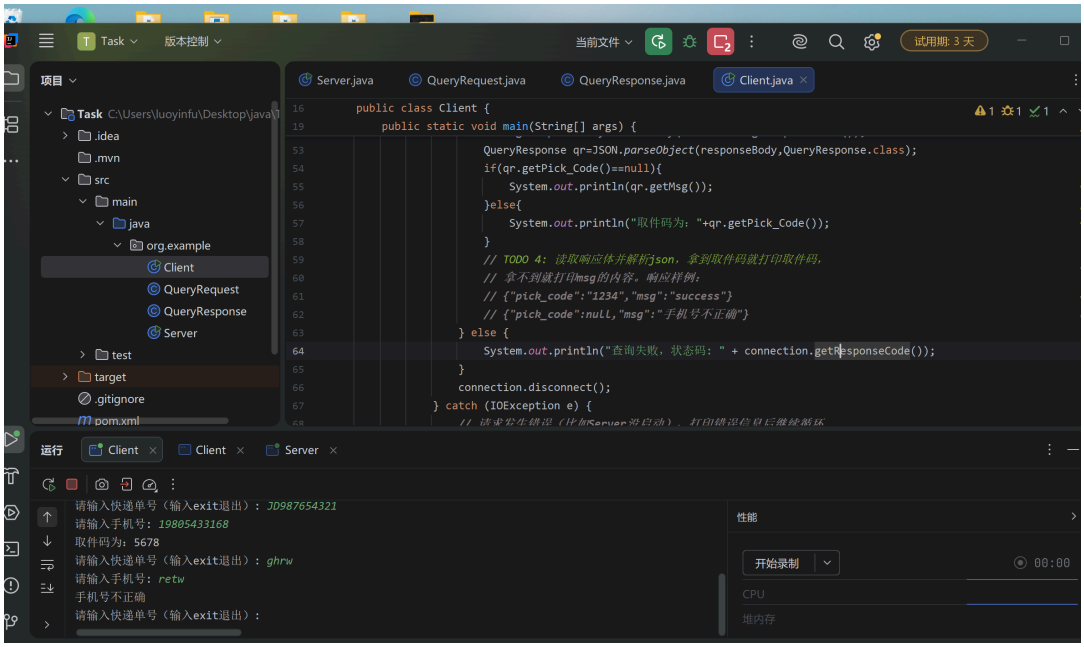

import java.io.*;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class Test {
    public static void main (String[] args) throws IOException {

        File file1=new File("name.txt");
        File file2=new File("name_sorted.txt");
        BufferedReader br=new BufferedReader
                (new InputStreamReader
                        (new FileInputStream(file1), StandardCharsets.
        BufferedWriter bw=new BufferedWriter
                (new OutputStreamWriter
                        (new FileOutputStream(file2), StandardCharsets
        String len;
        ArrayList<String> list=new ArrayList<>();
        while ((len=br.readLine())!=null) {
            if(len.isEmpty()){//判断 是否为空行
                continue;
            }
            list.add(len.trim());//去空格
        }
        br.close();//读取完毕，关闭输入缓冲流
        List<String> list1=list.stream().sorted().collect(Collectors.t
        for(String s:list1){
            bw.write(s);
            bw.newLine();
        }
        bw.close();//关闭输出缓冲流
    }
```

}

Task3




```
package org.example;

import com.alibaba.fastjson2.JSON;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.nio.charset.StandardCharsets;
import java.sql.SQLOutput;
import java.util.Scanner;

public class Client {
    private static final String SERVER_URL = "http://localhost:8000/qu
```

```
public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    while (true) {
        try {
            // 读取用户输入的快递单号
            System.out.print("请输入快递单号（输入exit退出）: ");
            String trackingNumber = scanner.nextLine().trim();
            if(trackingNumber.equals("exit")){
                break;
            }
            // TODO 1: 输入exit时退出循环（

            // 读取用户输入的手机号
            System.out.print("请输入手机号: ");
            String phone = scanner.nextLine().trim();

            // TODO 2: 用fastjson构造json格式的请求体，样例为
            // {"trackingNumber":"SF123456789","phone":"1300543367
            QueryRequest queryRequest = new QueryRequest(phone, tr
            String json = JSON.toJSONString(queryRequest);
            // TODO 3: 创建HttpURLConnection，把上面的json写入请求体
            // （注意：连接对象要命名为connection，下面判断状态码要用
            URL url=new URL(SERVER_URL);
            HttpURLConnection connection = (HttpURLConnection) url
            connection.setRequestMethod("POST");
            connection.setRequestProperty("Content-Type","applicat
            connection.setDoOutput(true);

            try(OutputStream os=connection.getOutputStream()){
                os.write(json.getBytes(StandardCharsets.UTF_8));
            }
            // 读取响应，状态码是200才是响应成功
            if (connection.getResponseCode() == 200) {
                String responseBody = readBody(connection.getInput
                QueryResponse qr=JSON.parseObject(responseBody,Que
                if(qr.getPick_Code()==null){
                    System.out.println(qr.getMsg());
                }else{
                    System.out.println("取件码为: "+qr.getPick_Code
                }
                // TODO 4: 读取响应体并解析json，拿到取件码就打印取件
                // 拿不到就打印msg的内容。响应样例：
                // {"pick_code":"1234","msg":"success"}
                // {"pick_code":null,"msg":"手机号不正确"}
            } else {
                System.out.println("查询失败，状态码: " + connectio
```

```
                }
                connection.disconnect();
            } catch (IOException e) {
                // 请求发生错误（比如Server没启动），打印错误信息后继续循
                System.out.println("请求发生错误: " + e.getMessage());
            }
        }
        scanner.close();
        System.out.println("客户端已退出");
    }

    /** 读取响应体：把InputStream按UTF-8读成一个字符串 */
    private static String readBody(InputStream is) throws IOException
        StringBuilder sb = new StringBuilder();
        try (BufferedReader reader = new BufferedReader(new InputStrea
            String line;
            while ((line = reader.readLine()) != null) {
                sb.append(line);
            }
        }
        return sb.toString();
    }
`
```

```
package org.example;

public class QueryRequest {
    private String trackingNumber;
    private String phone;

    public String getPhone() {
        return phone;
    }

    public void setPhone(String phone) {
        this.phone = phone;
    }

    public String getTrackingNumber() {
        return trackingNumber;
    }

    public void setTrackingNumber(String trackingNumber) {
        this.trackingNumber = trackingNumber;
    }
```

```java
    public QueryRequest(String phone, String trackingNumber) {
        this.phone = phone;
        this.trackingNumber = trackingNumber;
    }
    public QueryRequest() {
    }
}
```java
package org.example;

public class QueryResponse {

    private String pick_code;
    private String msg;
    public QueryResponse() {
    }

    public QueryResponse(String pick_code,String msg) {
        this.pick_code = pick_code;
        this.msg=msg;
    }

    public String getPick_Code() {
        return pick_code;
    }

    public void setPick_Code(String pick_code) {
        this.pick_code = pick_code;
    }

    public String getMsg() {
        return msg;
    }
    public void setMsg(String msg) {
        this.msg = msg;
    }
}



package org.example;

import com.alibaba.fastjson2.JSON;

import java.io.BufferedReader;
import java.io.IOException;
```
```

```java
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.nio.charset.StandardCharsets;
import java.sql.SQLOutput;
import java.util.Scanner;

public class Client {
    private static final String SERVER_URL = "http://localhost:8000/qu

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        while (true) {
            try {
                // 读取用户输入的快递单号
                System.out.print("请输入快递单号（输入exit退出）: ");
                String trackingNumber = scanner.nextLine().trim();
                if(trackingNumber.equals("exit")){
                    break;
                }
                // TODO 1: 输入exit时退出循环（

                // 读取用户输入的手机号
                System.out.print("请输入手机号: ");
                String phone = scanner.nextLine().trim();

                // TODO 2: 用fastjson构造json格式的请求体，样例为
                // {"trackingNumber":"SF123456789","phone":"1300543367
                QueryRequest queryRequest = new QueryRequest(phone, tr
                String json = JSON.toJSONString(queryRequest);
                // TODO 3: 创建HttpURLConnection，把上面的json写入请求体
                // （注意：连接对象要命名为connection，下面判断状态码要用
                URL url=new URL(SERVER_URL);
                HttpURLConnection connection = (HttpURLConnection) url
                connection.setRequestMethod("POST");
                connection.setRequestProperty("Content-Type","applicat
                connection.setDoOutput(true);

                try(OutputStream os=connection.getOutputStream()){
                    os.write(json.getBytes(StandardCharsets.UTF_8));
                }
                // 读取响应，状态码是200才是响应成功
                if (connection.getResponseCode() == 200) {
                    String responseBody = readBody(connection.getInput
                    QueryResponse qr=JSON.parseObject(responseBody,Que
```

```
                    if(qr.getPick_Code()==null){
                        System.out.println(qr.getMsg());
                    }else{
                        System.out.println("取件码为：“+qr.getPick_Code
                    }
                    // TODO 4: 读取响应体并解析json，拿到取件码就打印取件
                    // 拿不到就打印msg的内容。响应样例：
                    // {"pick_code":"1234","msg":"success"}
                    // {"pick_code":null,"msg":"手机号不正确"}
                } else {
                    System.out.println("查询失败，状态码：" + connectio
                }
                connection.disconnect();
            } catch (IOException e) {
                // 请求发生错误（比如Server没启动），打印错误信息后继续循
                System.out.println("请求发生错误：" + e.getMessage());
            }
        }
        scanner.close();
        System.out.println("客户端已退出");
    }

    /** 读取响应体：把InputStream按UTF-8读成一个字符串 */
    private static String readBody(InputStream is) throws IOException
        StringBuilder sb = new StringBuilder();
        try (BufferedReader reader = new BufferedReader(new InputStrea
            String line;
            while ((line = reader.readLine()) != null) {
                sb.append(line);
            }
        }
        return sb.toString();
    }
```

代码：

java

```java
private static final Map<String, String> exprMap = new HashMap<>();
```

✅ **一句话核心：**

`final` **修饰引用变量，只代表【引用地址不能改】，不代表里面容器的内容不能修改！**

```
            exchange.close();
        }
    }catch(Exception e){
        // 处理异常，返回400状态码(Bad Request)
        // TODO 6: 返回状态码400和错误JSON {"pick_code":null,"msg":"请求格式错误"}
            exchange.sendResponseHeaders( rCode: 400,  responseLength: -1);
            QueryResponse response = new QueryResponse( pickupCode: null, msg: "请求格式
            String json = JSON.toJSONString(response);
            exchange.getResponseBody().write(json.getBytes(StandardCharsets.UTF_8));
            exchange.close();
    }
}
```

对应你写的 HttpServer 代码

只要代码**成功执行到** `sendResponseHeaders(状态码, len)`，客户端就一定能拿到 HTTP 响应（状态码），后面还可以写响应 body。

- 400、200、500 都属于**HTTP 响应**，就算是错误码，也是正常 HTTP 响应。
- 只有代码抛出异常，**没执行 sendResponseHeaders**就直接 close 连接，客户端就收不到 HTTP 状态码。

极简一句话记忆

```
} else {
    // TODO 3: 用"单号_手机号"拼成key，在expressMap中查询取件码
    // TODO 4: 根据查询结果构造QueryResponse
    // TODO 5: 把响应对象序列化成JSON并发送响应（状态码200）
    String key = trackingNumber + "_" + phone;
    String pickCode = expressMap.get(key);
    QueryResponse response = new QueryResponse( pickCode, msg: "success");
    String json = JSON.toJSONString(response);
    exchange.sendResponseHeaders( rCode: 200, json.length());
    exchange.getResponseBody().write(json.getBytes(StandardCharsets.UTF_8));
    exchange.close();
```

//里面是字节数这样写如果中文就不行了

为什么这个坑很关键

sendResponseHeaders 第二个参数告诉客户端：**后面跟着多少字节的数据。**
如果你给错数字（传字符数而不是字节数）：

- 有中文时，客户端读到最后会发现字节数对不上，解析失败或者报截断。

✅ 一句话讲明白

对，`HttpServer` 就是**HTTP 服务端**，它会在你指定的 `PORT`（比如 8000 端口）**开启监听**，等着客户端来连接，就是你说的监听器。

拆解这几行代码

```java
// 创建服务器，绑定PORT端口，backlog=0是等待队列长度
HttpServer server = HttpServer.create(new InetSocketAddress(PORT), 0);
// 注册路由，访问 /query 这个路径，交给 QueryHandler 处理
server.createContext("/query", new QueryHandler());
// 正式启动，开始监听！
server.start();
```

1. `new InetSocketAddress(PORT)`：绑定本机的**PORT 端口**（比如 8000）。

   作用：操作系统把 8000 端口交给这个 Java 程序占用，所有发到本机 8000 端口的 HTTP 请求，都会交给这个 HttpServer。

2. `server.createContext("/query", new QueryHandler())`
   相当于**注册接口地址**：当请求路径是 `http://127.0.0.1:8000/query`，就进入 QueryHandler 的 handle 方法处理请求（就是你写的查询快递逻辑）。
3. `server.start()`：**开启监听**，启动监听器，开始等待客户端请求。

通俗类比

不完全是，这里要分清 JDK 自带 `HttpServer` 的规则：

1. **你写完 `sendResponseHeaders` + write 数据之后，数据已经发出去了**，但**连接不会结束。**
2. 客户端收到 TCP 数据，能读到内容，但**不知道「响应已经结束」**。

- HTTP 是基于流的，客户端没法知道：后面还有没有更多数据。
- 如果不调用 `exchange.close()`：客户端会**一直等待后续数据，不会认为本次请求完成**，请求挂起、请求超时。

简单一句话：

> 它**不是 Socket 连接本身，只是存【地址信息】的一个包装对象**（只存 IP + 端口，不负责收发数据） Oracle

作用

当你启动服务器、绑定端口、连接远程服务的时候，需要告诉程序：**连哪个 IP、哪个端口**，就用这个类包装。

构造方式（3 种常用写法）

```java
// 1. 直接指定主机名+端口
InetSocketAddress addr1 = new InetSocketAddress("127.0.0.1", 8080);

// 2. 只写端口，IP是0.0.0.0（本机所有网卡，启动http服务器常用！）
InetSocketAddress addr2 = new InetSocketAddress(8080);

// 3. InetAddress对象 + 端口
InetAddress ip = InetAddress.getByName("localhost");
InetSocketAddress addr3 = new InetSocketAddress(ip,8080);
```

容易搞混的 3 个类（一定要分清）

1. **InetAddress**：只代表 IP（127.0.0.1），不带端口
2. **InetSocketAddress**：**IP + 端口**（127.0.0.1:8080，地址信息）✅ 这个
3. **Socket / ServerSocket**：真正的套接字，**负责建立连接、收发数据**（通信通道）

通俗比喻：

exchange.sendResponseHeaders(500, -1); // 状态码改成500，-1代表不知道长度
一定要先梳理好逻辑，按照顺序写，先大体，后补充细节，在改进